



Figure 5: The Canny filter applied on an image



Figure 6: A test image

```
cv.WaitKey(3000)
cv.DestroyWindow("RGB")
```



Figure 7: Faces detected in the test image

Run it. The output should resemble Figure 3. From the ninth to the 11th line, we have described the parameters of the circle—its thickness, line type and co-ordinates. On Line 12, the `Circle` function is used, which has six parameters: the target image on which to draw the circle, the coordinates, the radius of the circle, the colour of the circle, the thickness and the line type. Other code is similar to previous examples.

The Laplacian of an image

Often, the Laplace operation is performed on an image to get the edges. The code below (*Laplace.py*) does this:

```
#!/usr/bin/python
import cv
src = cv.LoadImage("/home/jayneil/12.png")
print(src)
cv.NamedWindow("RGB", cv.CV_WINDOW_AUTOSIZE)
cv.ShowImage("RGB", src)
cv.WaitKey(1000)
cv.DestroyWindow("RGB")
dst = cv.CreateImage(cv.GetSize(src), cv.IPL_DEPTH_16S, 3)
laplace = cv.Laplace(src, dst)
cv.SaveImage("/home/jayneil/laplace.png", dst)
temp=cv.LoadImage("/home/jayneil/laplace.png")
cv.NamedWindow("Laplace", cv.CV_WINDOW_AUTOSIZE)
cv.ShowImage("Laplace", temp)
cv.WaitKey(2000)
cv.DestroyWindow("Laplace")
```

Figure 4 shows the expected output of running the code

given above. The `print` function outputs the properties of the image. On the 10th line, the `Laplace` function is used to obtain the edges. It takes two parameters—the source image and the variable in which to store the processed image. You can use `SaveImage` to save the resultant image, supplying the path, the file-name, and the variable containing the image. Then load the saved image and display it.

The Canny filter

This is a very popular filter, used to very accurately detect the edges in an image. The code below is *Canny.py*:

```
#!/usr/bin/python
import cv
src = cv.LoadImage("/home/jayneil/12.png")
cv.NamedWindow("RGB", cv.CV_WINDOW_AUTOSIZE)
cv.ShowImage("RGB", src)
cv.WaitKey(1000)
cv.DestroyWindow("RGB")
dst = cv.CreateImage(cv.GetSize(src), cv.IPL_DEPTH_8U, 1)
cv.CvtColor(src, dst, cv.CV_RGB2GRAY)
cv.NamedWindow("GRAY", cv.CV_WINDOW_AUTOSIZE)
cv.ShowImage("GRAY", dst)
cv.WaitKey(1000)
cv.DestroyWindow("GRAY")
dst1 = cv.CreateImage(cv.GetSize(src), cv.IPL_DEPTH_8U, 1)
cv.Canny(dst, dst1, 1.0, 4.0, aperture_size=3)
cv.NamedWindow("Canny_Filter", cv.CV_WINDOW_AUTOSIZE)
cv.ShowImage("Canny_Filter", dst1)
cv.WaitKey(4500)
cv.DestroyWindow("Canny_Filter")
```

Run the code and you should see something like what's shown in Figure 5. Convert the input image to grey-scale first (see the ninth line), before applying the Canny filter in the 15th line. This function takes five parameters: the source image, the variable in which to store the processed image, and the rest are the threshold values for this filter.

Face detection

Finally, let us detect faces in a still image, using a Haar classifier for detection. The Haar classifier is a pre-made program, which you can train to do operations based on your needs. The more you run the program, the more the Haar classifier trains itself. All Haar classifiers are located in `OpenCV-2.3.0/data/haarcascades`. The following is *Face.py*:

```
#!/usr/bin/python
import cv #import the openCV lib to python
a=cv.LoadImage("/home/jayneil/beautiful-faces.jpg")
print(a)
cv.NamedWindow("Original Image", cv.CV_WINDOW_AUTOSIZE)
cv.ShowImage("Original Image", a)
```